

# SBI 811: Computer Programming in Bioinformatics

## Lecture 6 — Dictionaries and Structured Biological Data

**Duration:** 3 hours

**Level:** Complete beginners

**Prerequisites:** strings, lists, conditions, loops and simple functions

**Biological applications:** nucleotide counts, sequence records, sample metadata, codon tables and simple DNA translation

Read each example, predict the output, run the cell, inspect the result, and then make one small deliberate change.

### Learning outcomes

By the end of this lecture, you should be able to:

1. explain why a dictionary is useful;
2. distinguish dictionary keys from values;
3. create an empty or populated dictionary;
4. retrieve a value using a key;
5. add and update key–value pairs;
6. use `in`, `get()`, `keys()`, `values()` and `items()`;
7. loop through a dictionary;
8. count nucleotides using a dictionary;
9. pass a dictionary to a function and return a dictionary;
10. represent one biological record with a dictionary;
11. work with a list of dictionaries;
12. access a nested dictionary carefully;
13. use a codon dictionary to translate a short DNA sequence; and
14. test dictionary-based biological code.

FASTA file reading, pandas, classes and external packages are intentionally postponed.

## Part 1 — Why do we need dictionaries?

A list stores values in an ordered collection. We access them using numerical positions.

### Begin with a mental picture

Imagine four labelled tubes in a laboratory. One tube is labelled **A**, another **T**, another **G**, and another **C**. Each tube contains a number representing how many times that nucleotide was observed.

A dictionary works in a similar way:

- the label on the tube is the **key**;
- the content associated with the label is the **value**.

For example, in `{"A": 4}`, the key is `"A"` and the value is `4`. The key gives meaning to the number. Without the key, the number `4` tells us very little.

Lists and dictionaries solve different problems. A list is most natural when position matters: first, second, third. A dictionary is most natural when a meaningful label should identify each value.

```
In [ ]: nucleotide_counts_list = [4, 3, 2, 1]

print(nucleotide_counts_list)
print("First value:", nucleotide_counts_list[0])
```

The list does not explain whether `4` is the count for `A`, `T`, `G` or `C`. We must remember the agreed order.

A **dictionary** connects each value to a meaningful label called a **key**.

```
In [ ]: nucleotide_counts = {
    "A": 4,
    "T": 3,
    "G": 2,
    "C": 1
}

print(nucleotide_counts)
print("A count:", nucleotide_counts["A"])
```

A dictionary stores **key–value pairs**:

| Key | Value |
|-----|-------|
| "A" | 4     |
| "T" | 3     |
| "G" | 2     |
| "C" | 1     |

Think of a dictionary as a labelled storage system: give Python a key and it retrieves the associated value.

## 1.1 Dictionary syntax

```
variable_name = {
    key_1: value_1,
```

```
key_2: value_2
}
```

- curly braces `{}` surround the dictionary;
- a colon separates each key from its value;
- a comma separates pairs;
- keys must be unique;
- values may be repeated; and
- indentation across several lines improves readability.

## Reading dictionary punctuation slowly

Consider one pair:

```
"sample_id": "S01"
```

Read the colon as **maps to** or **has the value**:

The key `sample_id` maps to the value `S01`.

Now consider two pairs:

```
{"sample_id": "S01", "organism": "Rhizobium"}
```

The comma ends the first pair and begins the next. The surrounding curly braces tell Python that the complete object is a dictionary.

The quotation marks are required here because the keys and values are strings. Numeric or Boolean values do not use quotation marks:

```
{"length": 850, "passed_qc": True}
```

If `850` were written as `"850"`, Python would treat it as text rather than a number.

```
In [ ]: sample = {
        "sample_id": "S01",
        "organism": "Rhizobium",
        "sequence_length": 850,
        "passed_qc": True
    }

    print(sample)
    print(type(sample))
```

A dictionary can contain values of different types. Here the values include strings, an integer and a Boolean.

## 1.2 An empty dictionary

An empty dictionary is often created before a loop fills it.

## Why begin with an empty dictionary?

At the beginning of an analysis, the program may not yet know which symbols or categories it will encounter. An empty dictionary provides a container that can be filled as the program processes data.

```
results = {}
```

This does not mean the analysis has failed. It means the result structure exists but currently contains zero pairs. Later, code such as `results["A"] = 3` can add information.

Compare the two uses of `len()` :

- `len("ATGC")` gives the number of characters in a string;
- `len({"A": 2, "T": 2})` gives the number of key–value pairs in a dictionary.

The meaning of a function can therefore depend on the type of object supplied to it.

```
In [ ]: results = {}  
print(results)  
print(len(results))
```

`len(dictionary)` reports the number of key–value pairs, not the length of the values.

## Guided practice 1 — Create a sequence dictionary

Create a dictionary named `sequence_record` containing:

- sequence ID: `gene_001`
- sequence: `ATGCGTAA`
- organism: `Rhizobium leguminosarum`
- circular: `False`

Then print the whole dictionary.

```
In [ ]: # Write your dictionary here.
```

## Guided practice 1 — solution

```
In [ ]: sequence_record = {  
    "id": "gene_001",  
    "sequence": "ATGCGTAA",  
    "organism": "Rhizobium leguminosarum",  
    "circular": False  
}  
  
print(sequence_record)
```

## Part 2 — Accessing values

Use square brackets containing a key.

## Follow the lookup operation

Suppose Python sees:

```
sequence_record["organism"]
```

It performs these conceptual steps:

1. find the dictionary named `sequence_record` ;
2. look among its keys for the exact string `"organism"` ;
3. retrieve the value associated with that key;
4. return that value to the surrounding expression.

The key is not a variable position. Python is not being asked for the first or second item. It is being asked for the item with a particular label.

This is why `sequence_record[0]` normally fails: unless the dictionary literally has the integer `0` as a key, there is no entry with that label.

```
In [ ]: print(sequence_record["id"])
        print(sequence_record["sequence"])
        print(sequence_record["organism"])
        print(sequence_record["circular"])
```

Dictionary access uses a key rather than a numerical position.

`sequence_record["sequence"]` means “retrieve the value stored under the key `sequence` .”

## 2.1 Keys are exact

Keys are case-sensitive. `"id"`, `"ID"` and `"Id"` are different keys.

Trying to retrieve a missing key using square brackets raises `KeyError` . The demonstration catches the error so the notebook continues.

## Why `KeyError` is useful information

`KeyError` means that the dictionary exists, but the requested key does not. Common causes include:

- spelling the key incorrectly;
- using different capitalization;
- assuming an optional field is always present; or
- running notebook cells in the wrong order.

For example, `"length"` , `"Length"` and `"sequence_length"` are three different keys. Python does not guess that they may refer to the same biological measurement.

When you see `KeyError` , first print the available keys:

```
print(record.keys())
```

Then compare the requested key character by character with the available keys.

```
In [ ]: try:
        print(sequence_record["length"])
        except KeyError as error:
            print("KeyError:", error)
```

## 2.2 Safe lookup with `get()`

`.get()` returns the value when the key exists. If it is absent, it returns `None` instead of stopping the program.

## Required fields and optional fields

Choosing between square brackets and `.get()` expresses an assumption about the data.

An ID may be required:

```
sample_id = record["id"]
```

If the ID is absent, stopping with an error can be useful because the record is incomplete.

A note may be optional:

```
note = record.get("note", "No note supplied")
```

Here, absence is expected and a default value allows the program to continue.

Do not use `.get()` merely to hide every error. If a field is scientifically required, silently replacing it with a default may conceal poor-quality data.

```
In [ ]: print(sequence_record.get("sequence"))
        print(sequence_record.get("length"))
```

A default value can be supplied.

```
In [ ]: print(sequence_record.get("length", "Length not recorded"))
        print(sequence_record.get("organism", "Unknown organism"))
```

## Square brackets or `.get()` ?

| Approach                        | Missing key                                   | Appropriate when                                    |
|---------------------------------|-----------------------------------------------|-----------------------------------------------------|
| <code>record["id"]</code>       | raises <code>KeyError</code>                  | the key is required and absence indicates a problem |
| <code>record.get("note")</code> | returns <code>None</code> or a chosen default | the key is optional                                 |

Neither approach is always superior. Choose based on what the data should contain.

## Guided practice 2 — Retrieve information

From `sequence_record` :

1. print the ID using square brackets;
2. print the organism using `.get()` ;
3. request an optional `description` key and display `No description` if missing.

```
In [ ]: # Write your code here.
```

## Guided practice 2 — solution

```
In [ ]: print(sequence_record["id"])
        print(sequence_record.get("organism"))
        print(sequence_record.get("description", "No description"))
```

## Part 3 — Adding and updating values

Assignment with a new key adds a pair.

### Dictionaries are mutable

**Mutable** means that the object can be changed after it is created. When this statement runs:

```
sequence_record["length"] = 8
```

Python does not create a completely unrelated record. It modifies the dictionary already stored under the name `sequence_record`.

The same assignment syntax has two possible effects:

- if the key is absent, a new pair is added;
- if the key already exists, its old value is replaced.

Therefore, always check the spelling of a key before assignment. Accidentally using `"lenght"` will not correct `"length"` ; it will create an additional misspelled key.

```
In [ ]: sequence_record["length"] = 8
        print(sequence_record)
```

Assignment with an existing key replaces its value.

```
In [ ]: sequence_record["organism"] = "Rhizobium sp."
        print(sequence_record["organism"])
```

**Important:** dictionaries are mutable. Updating a key changes the existing dictionary.

## 3.1 Calculate rather than type derived values

The length should be calculated from the sequence, not typed manually.

### Raw fields and derived fields

It helps to distinguish two kinds of information:

- **raw fields** are supplied with the observation, such as an ID or sequence;
- **derived fields** are calculated from raw information, such as length or GC percentage.

Typing `"length": 8` manually creates a risk that the sequence will later change while the recorded length remains 8. Calculating the value keeps the fields consistent:

```
record["length"] = len(record["sequence"])
```

Read the expression from inside outward:

1. retrieve the sequence;
2. calculate its length;
3. store that result under the key `length`.

```
In [ ]: sequence_record["length"] = len(sequence_record["sequence"])
print(sequence_record["length"])
```

## 3.2 Add a GC percentage

We can reuse a function from Lecture 5.

```
In [ ]: def calculate_gc_percentage(sequence):
        """Return GC percentage; return 0.0 for an empty sequence."""
        cleaned_sequence = sequence.strip().upper()
        if len(cleaned_sequence) == 0:
            return 0.0
        gc_count = cleaned_sequence.count("G") + cleaned_sequence.count("C")
        return (gc_count / len(cleaned_sequence)) * 100

        sequence_record["gc_percentage"] = calculate_gc_percentage(
            sequence_record["sequence"]
        )

        print(sequence_record)
```

## 3.3 Removing a key

`pop()` removes a key and returns its value. Use it only when removal is intended.

```
In [ ]: temporary_record = {
        "id": "S01",
        "sequence": "ATGC",
        "temporary_note": "remove later"
    }

    removed_value = temporary_record.pop("temporary_note")
    print("Removed:", removed_value)
    print("Remaining:", temporary_record)
```

This lesson focuses mainly on adding, updating and reading. Avoid deleting information from real scientific data unless the reason is clear and recorded.

# Part 4 — Membership testing

The `in` operator checks dictionary **keys** by default.

## What exactly does `in` examine?

For a string, `in` checks characters or substrings. For a list, it checks list elements. For a dictionary, it checks **keys** unless another dictionary view is specified.

`"sequence" in record`

asks whether `sequence` is one of the keys. It does not ask whether the letters of a DNA sequence occur somewhere in the record.

To make the intention explicit:

- `value in record` checks keys;
- `value in record.keys()` also checks keys;
- `value in record.values()` checks values.

Checking values is sometimes helpful, but in scientific records it is usually clearer to retrieve a known field and examine that field directly.

```
In [ ]: print("sequence" in sequence_record)
        print("description" in sequence_record)
        print("ATGCGTAA" in sequence_record)
```

The last expression is `False` because `"ATGCGTAA"` is a value, not a key.

```
In [ ]: print("ATGCGTAA" in sequence_record.values())
```

## 4.1 Check before access

```
In [ ]: if "description" in sequence_record:
        print(sequence_record["description"])
        else:
            print("Description is not available.")
```

## Guided practice 3 — Add only when missing

Check whether `quality` is a key in `sequence_record`. If it is missing, add `"quality": "not assessed"`. Then print the dictionary.

```
In [ ]: # Write your code here.
```

## Guided practice 3 — solution

```
In [ ]: if "quality" not in sequence_record:
        sequence_record["quality"] = "not assessed"
```

```
print(sequence_record)
```

## Part 5 — Dictionary methods

Three important methods provide different views:

- `.keys()` — all keys;
- `.values()` — all values;
- `.items()` — key–value pairs.

### Views rather than new biological results

The objects returned by `.keys()`, `.values()` and `.items()` are different ways of viewing the same dictionary. They do not calculate new measurements.

Suppose the dictionary is:

```
{"A": 3, "T": 2}
```

Then conceptually:

- `.keys()` provides `A` and `T`;
- `.values()` provides `3` and `2`;
- `.items()` provides the pairs `(A, 3)` and `(T, 2)`.

Use the view that matches the question:

- “Which categories exist?” → keys;
- “What measurements were recorded?” → values;
- “Which measurement belongs to each category?” → items.

```
In [ ]: print(sequence_record.keys())
        print(sequence_record.values())
        print(sequence_record.items())
```

These views are especially useful in loops.

### 5.1 Loop through keys

```
In [ ]: for key in sequence_record:
        print(key)
```

Writing `for key in sequence_record` loops through keys. The explicit form below means the same thing.

```
In [ ]: for key in sequence_record.keys():
        print(key)
```

### 5.2 Loop through values

```
In [ ]: for value in sequence_record.values():  
        print(value)
```

## 5.3 Loop through keys and values together

### Understanding two loop variables

In this loop:

```
for key, value in sequence_record.items():
```

`.items()` supplies one pair on each iteration. Python separates, or **unpacks**, the pair into two variables.

On one iteration, the pair may be:

```
("id", "gene_001")
```

Python assigns `"id"` to `key` and `"gene_001"` to `value`. The variable names are chosen by the programmer; the following would also run:

```
for field_name, field_value in sequence_record.items():
```

The second names may be easier for a beginner to interpret because they describe the role of each variable.

```
In [ ]: for key, value in sequence_record.items():  
        print(key, "->", value)
```

On each iteration, `.items()` supplies one pair. Python stores the first part in `key` and the second in `value`.

### Guided practice 4 — Format a report

Loop through `sequence_record.items()` and display each pair in this style:

```
id: gene_001  
sequence: ATGCGTAA
```

Do not print the dictionary directly.

```
In [ ]: # Write your loop here.
```

### Guided practice 4 — solution

```
In [ ]: for key, value in sequence_record.items():  
        print(f"{key}: {value}")
```

## Part 6 — Counting nucleotides with a dictionary

In the consolidation practical, four separate variables were used:

```
a_count = 0
t_count = 0
g_count = 0
c_count = 0
```

A dictionary keeps related counters together.

## Trace the counting algorithm by hand

For the short sequence `ATA`, begin with:

```
{"A": 0, "T": 0, "G": 0, "C": 0}
```

The dictionary changes after each loop iteration:

| Nucleotide read | A | T | G | C |
|-----------------|---|---|---|---|
| before the loop | 0 | 0 | 0 | 0 |
| first A         | 1 | 0 | 0 | 0 |
| T               | 1 | 1 | 0 | 0 |
| final A         | 2 | 1 | 0 | 0 |

Only one key is updated during each iteration. The dictionary remembers the previous counts, which is why the final result represents the entire sequence.

Tracing a very short input by hand is one of the best ways to understand or debug a loop.

```
In [ ]: dna = "ATGCGTAACGTTAGC"

counts = {
    "A": 0,
    "T": 0,
    "G": 0,
    "C": 0
}

for base in dna:
    counts[base] = counts[base] + 1

print(counts)
```

## 6.1 Understand the update carefully

```
counts[base] = counts[base] + 1
```

If `base` is `G`, Python reads the current value under `"G"`, adds one, and stores the new value back under `"G"`.

The shorter form is:

## Assignment uses the old value to create the new value

The right side of an assignment is evaluated first:

```
counts[base] = counts[base] + 1
```

If `base` currently contains `"G"` and the stored G count is 2:

1. `counts[base]` on the right retrieves 2;
2. Python calculates `2 + 1`;
3. the result 3 is assigned back to the G key.

The shorter form `counts[base] += 1` performs the same update. Beginners should first understand the longer form; the shorter form is convenient once the reasoning is clear.

```
In [ ]: counts = {"A": 0, "T": 0, "G": 0, "C": 0}

for base in dna:
    counts[base] += 1

print(counts)
```

## 6.2 What if an unexpected symbol occurs?

The direct update fails if the symbol is not already a key.

```
In [ ]: dna_with_n = "ATGCNTA"
counts = {"A": 0, "T": 0, "G": 0, "C": 0}

try:
    for base in dna_with_n:
        counts[base] += 1
except KeyError as error:
    print("Unexpected symbol:", error)
```

One design is to include expected unknown symbols.

```
In [ ]: counts = {"A": 0, "T": 0, "G": 0, "C": 0, "N": 0}

for base in dna_with_n:
    counts[base] += 1

print(counts)
```

Another design uses `get()` so new symbols can be counted dynamically.

## How dynamic counting differs from fixed counting

A fixed dictionary begins with the allowed categories. This is useful when the categories are known and unexpected symbols should be rejected.

A dynamic dictionary begins empty and creates a key for every observed symbol. This is useful when the possible categories are not known in advance.

The expression:

```
observed_counts.get(base, 0)
```

means: retrieve the existing count for this base, but use zero if it has never been seen. After adding one, the assignment creates or updates the appropriate key.

In bioinformatics, dynamic counting and validation should often be separate. A program may count that `X` occurs twice, while a validation step explains that `X` is not accepted in the chosen DNA alphabet.

```
In [ ]: observed_counts = {}

for base in "ATGCNXA":
    observed_counts[base] = observed_counts.get(base, 0) + 1

print(observed_counts)
```

Read the update as:

1. retrieve the current count for `base` ;
2. use zero if the key is absent;
3. add one; and
4. store the new count.

This is flexible, but it does not validate the biological meaning of a character. Counting and validation are different tasks.

## 6.3 Put counting inside a function

### Why return the dictionary?

If the function only printed counts, another part of the program could not easily reuse them. Returning the dictionary makes the result available for further work:

```
counts = count_symbols(sequence)
g_count = counts.get("G", 0)
```

The local dictionary is created afresh each time the function is called. This is important: counts from one sequence should not accidentally carry into the next sequence.

The function performs three separate stages:

1. standardize the input;
2. build the counts during a loop;
3. return the completed data structure.

```
In [ ]: def count_symbols(sequence):
        """Return a dictionary containing the count of every observed symbol."""
        counts = {}
        cleaned_sequence = sequence.strip().upper()

        for symbol in cleaned_sequence:
            counts[symbol] = counts.get(symbol, 0) + 1

        return counts
```

```
print(count_symbols("ATGCGTAA"))
print(count_symbols("atgna"))
```

## Testing the counting function

```
In [ ]: assert count_symbols("AAAA") == {"A": 4}
assert count_symbols("ATGC") == {"A": 1, "T": 1, "G": 1, "C": 1}
assert count_symbols("") == {}
assert count_symbols("atga") == {"A": 2, "T": 1, "G": 1}

print("All count tests passed.")
```

## Guided practice 5 — Count amino-acid symbols

Use `count_symbols()` with the protein sequence:

MKTLLVAVAV

Store the returned dictionary and print each amino acid and count using `.items()`.

```
In [ ]: protein = "MKTLLVAVAV"

# Write your code here.
```

## Guided practice 5 — solution

```
In [ ]: protein = "MKTLLVAVAV"
amino_acid_counts = count_symbols(protein)

for amino_acid, count in amino_acid_counts.items():
    print(amino_acid, count)
```

## Part 7 — A function that returns a standard nucleotide dictionary

Sometimes we want all four bases present even when a count is zero.

### Standard shape versus observed shape

`count_symbols("AAAA")` returns only the symbol that was observed:

```
{"A": 4}
```

`count_dna_bases("AAAA")` returns a standard DNA structure:

```
{"A": 4, "T": 0, "G": 0, "C": 0}
```

Neither structure is universally correct. The standard structure can make later comparisons easier because all records contain the same four keys. The observed structure is more flexible because it can represent any symbol.

The appropriate design depends on what later stages of the analysis expect.

```
In [ ]: def count_dna_bases(sequence):
        """Return A, T, G and C counts for a valid DNA sequence."""
        cleaned_sequence = sequence.strip().upper()
        counts = {"A": 0, "T": 0, "G": 0, "C": 0}

        for base in cleaned_sequence:
            if base not in counts:
                return None
            counts[base] += 1

        return counts

print(count_dna_bases("ATGCGTAA"))
print(count_dna_bases("AAAA"))
print(count_dna_bases("ATGX"))
```

Here `None` signals that the sequence contains a symbol the function does not accept. In larger software, the chosen error behaviour must be clearly documented.

```
In [ ]: assert count_dna_bases("AAAA") == {"A": 4, "T": 0, "G": 0, "C": 0}
assert count_dna_bases("ATGC") == {"A": 1, "T": 1, "G": 1, "C": 1}
assert count_dna_bases("ATGX") is None
assert count_dna_bases("") == {"A": 0, "T": 0, "G": 0, "C": 0}

print("DNA-count tests passed.")
```

## Part 8 — Dictionaries as biological records

A dictionary can represent one sequence record with several labelled fields.

### A record represents one biological object

A sequence record groups facts that belong together. The ID, organism and sequence are not independent measurements floating in the notebook; they describe the same biological record.

Using a dictionary reduces the risk of mixing information from different samples. Compare:

```
ids = ["S01", "S02"]
sequences = ["ATGC", "GGCC"]
```

Here, the programmer must preserve matching positions across two lists. With dictionaries, the relationship is explicit:

```
{"id": "S01", "sequence": "ATGC"}
```

This structure is still small enough to inspect directly, which makes it suitable for beginner exercises.

```
In [ ]: record = {
        "id": "seq_01",
        "organism": "Escherichia coli",
        "sequence": "ATGCGTAA",
        "source": "chromosome"
    }
```

```
record["length"] = len(record["sequence"])
record["gc_percentage"] = calculate_gc_percentage(record["sequence"])
record["base_counts"] = count_dna_bases(record["sequence"])

for key, value in record.items():
    print(f"{key}: {value}")
```

The value under `base_counts` is itself a dictionary. This is a **nested dictionary**.

## Reading nested access from the outside inward

For:

```
record["base_counts"]["G"]
```

do not try to understand the complete expression at once. Break it into stages:

```
inner_dictionary = record["base_counts"]
g_count = inner_dictionary["G"]
```

The one-line and two-line versions mean the same thing. The two-line version is often easier to debug because the intermediate object can be printed.

If an error occurs, ask which level is missing: does the outer record lack `base_counts`, or does the inner dictionary lack `G`?

```
In [ ]: print(record["base_counts"])
        print(record["base_counts"]["G"])
```

Read the second expression from left to right:

1. `record["base_counts"]` retrieves the inner dictionary;
2. `["G"]` retrieves the `G` count from that inner dictionary.

## 8.1 Avoid overly deep nesting

Nested structures are useful, but too many levels become difficult for beginners to read and debug. Use a simple structure unless greater complexity is genuinely needed.

## Guided practice 6 — Enrich a record

Add these derived fields to the dictionary below:

- `length`
- `gc_percentage`
- `base_counts`
- `contains_start_codon` — `True` if the sequence starts with `ATG`

Then print a formatted report.

```
In [ ]: gene_record = {
        "id": "gene_A",
        "sequence": "ATGAAACCGTAA",
```

```
"organism": "Rhizobium sp."
}

# Add the derived fields and print the report.
```

## Guided practice 6 — solution

```
In [ ]: gene_record = {
        "id": "gene_A",
        "sequence": "ATGAAACCGTAA",
        "organism": "Rhizobium sp."
    }

gene_record["length"] = len(gene_record["sequence"])
gene_record["gc_percentage"] = calculate_gc_percentage(gene_record["sequence"])
gene_record["base_counts"] = count_dna_bases(gene_record["sequence"])
gene_record["contains_start_codon"] = gene_record["sequence"].startswith("ATG")

for key, value in gene_record.items():
    print(f"{key}: {value}")
```

## Part 9 — A list of dictionaries

A list can store several records, while each dictionary labels the fields of one record.

### Rows and fields

A list of dictionaries can be imagined as a small table:

- each dictionary is one row or record;
- each key is a field name or column label;
- each value is the observation for that record and field.

However, this is still ordinary Python, not pandas. We process the records with loops.

```
for sample in samples:
```

On each iteration, `sample` refers to one complete dictionary. Therefore

`sample["sequence"]` retrieves the sequence belonging to the current record, not an unrelated sequence.

```
In [ ]: samples = [
        {"id": "S01", "sequence": "ATGCGTAA", "site": "Kenya"},
        {"id": "S02", "sequence": "GGCCGGCC", "site": "Senegal"},
        {"id": "S03", "sequence": "AAAATTTT", "site": "South Africa"}
    ]

print(samples)
print(samples[0])
print(samples[0]["id"])
```

`samples[0]["id"]` means:

1. retrieve the first list item;
2. that item is a dictionary;

3. retrieve its `id` value.

## 9.1 Process every record

### How the loop enriches each record

The processing loop takes one original record and adds derived fields to that same record.

For each sample it:

1. retrieves the sequence;
2. calculates its length;
3. calculates GC percentage;
4. stores both results under new keys.

After the loop, the `samples` list still contains the same number of records, but each dictionary contains more information.

This pattern—read a record, calculate new information, attach the result—is common in data analysis. It should be used carefully so that the meaning and units of every new field remain clear.

```
In [ ]: for sample in samples:
        sequence = sample["sequence"]
        sample["length"] = len(sequence)
        sample["gc_percentage"] = calculate_gc_percentage(sequence)

        for sample in samples:
            print(
                sample["id"],
                sample["site"],
                sample["length"],
                round(sample["gc_percentage"], 2)
            )
```

## 9.2 Select records using a condition

```
In [ ]: for sample in samples:
        if sample["gc_percentage"] >= 60:
            print(sample["id"], "has high GC content")
```

## Guided practice 7 — Summarize sample records

For every sample:

1. add `base_counts` ;
2. add `gc_category` using low, moderate and high boundaries from Lecture 4;
3. print the ID, site and category.

Try before viewing the solution.

```
In [ ]: # Work with the existing samples list.
        # Write your code here.
```

## Guided practice 7 — solution

```
In [ ]: for sample in samples:
        sample["base_counts"] = count_dna_bases(sample["sequence"])

        gc_percentage = sample["gc_percentage"]
        if gc_percentage >= 60:
            sample["gc_category"] = "high"
        elif gc_percentage >= 40:
            sample["gc_category"] = "moderate"
        else:
            sample["gc_category"] = "low"

        print(sample["id"], sample["site"], sample["gc_category"])
```

## Part 10 — Codons as dictionary keys

A codon contains three nucleotides. A genetic-code dictionary can connect a codon key to an amino-acid value.

We will begin with a deliberately small table so the logic remains clear.

### Why a dictionary fits a genetic code lookup

A lookup table answers a direct question:

Given this codon, what amino-acid symbol should be returned?

The codon is therefore a natural key and the amino acid is its value:

"ATG": "M"

Several codon keys may have the same value because the genetic code is degenerate. For example, TTT and TTC can both map to F. This is allowed because values may repeat; only keys must be unique.

The teaching table is incomplete. It demonstrates the computational relationship without claiming to contain the complete standard genetic code.

```
In [ ]: small_codon_table = {
        "ATG": "M",
        "TTT": "F",
        "TTC": "F",
        "GCT": "A",
        "GCC": "A",
        "TAA": "STOP",
        "TAG": "STOP",
        "TGA": "STOP"
    }

    print(small_codon_table["ATG"])
```

```
print(small_codon_table["TAA"])
print(small_codon_table.get("CCC", "Unknown codon"))
```

## 10.1 Translate codons one at a time

```
In [ ]: codons = ["ATG", "GCT", "TTT", "TAA"]

for codon in codons:
    amino_acid = small_codon_table.get(codon, "?")
    print(codon, "->", amino_acid)
```

## 10.2 Split DNA into codons and look them up

The sequence is read from position zero in steps of three.

### Follow the translation loop in stages

With a step size of three, `range(0, len(dna), 3)` produces candidate starting positions 0, 3, 6 and so on. At each position:

1. `dna[position:position + 3]` extracts a codon;
2. the length check rejects an incomplete final piece;
3. the dictionary lookup converts the codon to an amino-acid symbol;
4. a stop value ends translation;
5. otherwise, the amino-acid symbol is added to the protein string.

The computer does not “understand” translation biologically. It follows the rules supplied by the programmer. If the reading frame, codon table or stop rule is wrong, the program may run normally while producing a biologically misleading result.

```
In [ ]: dna = "ATGGCTTTTAA"
protein = ""

for position in range(0, len(dna), 3):
    codon = dna[position:position + 3]

    if len(codon) < 3:
        print("Incomplete codon ignored:", codon)
        continue

    amino_acid = small_codon_table.get(codon, "?")

    if amino_acid == "STOP":
        break

    protein = protein + amino_acid

print("DNA:", dna)
print("Protein:", protein)
```

## Biological and computational cautions

This teaching example simplifies translation:

- it assumes reading begins at position zero;
- it uses only a small codon table;
- `?` indicates a codon absent from the table;
- translation stops at the first listed stop codon;
- it does not search for genes or choose a reading frame; and
- it does not model alternative genetic codes.

**A program can run and still use an incorrect biological assumption.** Always state the assumptions behind a result.

## 10.3 Put translation in a function

### Two inputs make the function reusable

The function receives both `dna` and `codon_table`. It does not depend on one fixed global table hidden elsewhere in the notebook.

This design allows the caller to supply:

- a small teaching table;
- an expanded standard table; or
- a table representing a different genetic code.

The function's behaviour is therefore partly determined by its data input. This is an important programming idea: a general algorithm can work with different lookup information without rewriting its control structure.

```
In [ ]: def translate_with_table(dna, codon_table):
        """Translate DNA from position zero using the supplied codon table."""
        cleaned_dna = dna.strip().upper()
        protein = ""

        for position in range(0, len(cleaned_dna), 3):
            codon = cleaned_dna[position:position + 3]

            if len(codon) < 3:
                break

            amino_acid = codon_table.get(codon, "?")

            if amino_acid == "STOP":
                break

            protein = protein + amino_acid

        return protein

print(translate_with_table("ATGGCTTTTAA", small_codon_table))
```

### Test translation with known inputs

```
In [ ]: assert translate_with_table("ATG", small_codon_table) == "M"
assert translate_with_table("ATGGCT", small_codon_table) == "MA"
assert translate_with_table("ATGTAA", small_codon_table) == "M"
assert translate_with_table("", small_codon_table) == ""
assert translate_with_table("atgttt", small_codon_table) == "MF"

print("Translation tests passed.")
```

## Guided practice 8 — Interpret unknown codons

Run the translation function with `ATGCCCTAA`.

1. What protein string is produced?
2. Why?
3. Add `CCC` to a copy of the small codon table with value `P`.
4. Run the translation again.
5. Explain why modifying the lookup table changes the result without changing the translation function.

```
In [ ]: # Write your investigation here.
```

## Guided practice 8 — solution

```
In [ ]: print(translate_with_table("ATGCCCTAA", small_codon_table))

extended_codon_table = small_codon_table.copy()
extended_codon_table["CCC"] = "P"

print(translate_with_table("ATGCCCTAA", extended_codon_table))
```

## Part 11 — Copying dictionaries

This assignment does not make an independent dictionary:

### Two names can point to one object

After:

```
alias = original
```

there are two names but only one dictionary object. Changing the object through either name makes the change visible through both names.

An analogy is placing two labels on the same specimen tube. Renaming the reference does not produce a second specimen.

`.copy()` produces a separate outer dictionary. This distinction matters when an original record must be preserved while an experimental version is modified.

```
In [ ]: original = {"A": 1, "T": 2}
alias = original
```

```
alias["A"] = 99

print("Original:", original)
print("Alias:", alias)
```

Both names refer to the same mutable object. Use `.copy()` for a separate shallow copy.

```
In [ ]: original = {"A": 1, "T": 2}
        independent_copy = original.copy()
        independent_copy["A"] = 99

        print("Original:", original)
        print("Copy:", independent_copy)
```

For the simple, non-nested dictionaries used here, `.copy()` is enough. Nested mutable structures require more care, which is beyond this lecture.

# Part 12 — Common mistakes

## Mistake 1: Using a missing key

```
record["species"]
```

Use the correct key, test membership, or use `.get()` when the field is optional.

## Mistake 2: Confusing a key with a value

```
"ATGC" in record
```

This checks keys. Use `"ATGC" in record.values()` to search values.

## Mistake 3: Forgetting that keys are case-sensitive

```
"id" and "ID" are different.
```

## Mistake 4: Replacing a whole dictionary accidentally

```
counts = counts["A"]
```

Now `counts` holds one number rather than the dictionary.

## Mistake 5: Changing a dictionary while looping over its keys

Avoid adding or removing keys during a simple key loop. Plan the update separately.

## Mistake 6: Assuming counted symbols are biologically valid

A flexible counter may count `X`, but that does not make `X` a valid DNA base.

## A practical dictionary-debugging checklist

When dictionary code fails or produces an unexpected result, ask these questions in order:

1. Is the object actually a dictionary? Check with `type()`.
2. What keys are currently available? Print `.keys()`.
3. Is the requested key spelled and capitalized exactly?
4. Is the value the expected type? Check with `type(record[key])`.
5. Did an earlier assignment replace the dictionary or one of its values?
6. Did two variable names accidentally refer to the same mutable dictionary?
7. Does the dictionary structure match the biological question?

This separates Python-structure errors from biological-interpretation errors.

# Part 13 — Integrated biological workflow

The program below combines lists, dictionaries, functions, loops and conditions. It:

1. cleans each sequence;
2. builds a dictionary record;
3. validates and counts DNA bases;
4. adds derived values;
5. stores every record in a list; and
6. prints a report.

## Read the workflow as a pipeline

The integrated code is longer because it coordinates several tasks, not because dictionaries themselves are mysterious.

The major stages are:

```
raw tuple
↓
clean sequence
↓
create record dictionary
↓
validate sequence
↓
add either results or rejection reason
↓
append completed record to the results list
↓
print the records
```

Focus on one stage at a time. The record dictionary acts as the object passed conceptually from one stage to the next. Every result remains connected to the correct sample ID and source.

```
In [ ]: raw_samples = [
        ("S01", " atgcgtaa ", "root"),
        ("S02", "GGCCGGCC", "nodule"),
        ("S03", "ATGXGTAA", "soil"),
        ("S04", "AAAATTTT", "leaf")
    ]

analysis_records = []

for sample_id, raw_sequence, source in raw_samples:
    sequence = raw_sequence.strip().upper()
    base_counts = count_dna_bases(sequence)

    record = {
        "id": sample_id,
        "source": source,
        "sequence": sequence,
```

```

        "length": len(sequence)
    }

    if base_counts is None:
        record["valid"] = False
        record["reason"] = "contains a non-ATGC symbol"
    else:
        record["valid"] = True
        record["base_counts"] = base_counts
        record["gc_percentage"] = calculate_gc_percentage(sequence)

        if record["gc_percentage"] >= 60:
            record["gc_category"] = "high"
        elif record["gc_percentage"] >= 40:
            record["gc_category"] = "moderate"
        else:
            record["gc_category"] = "low"

    analysis_records.append(record)

for record in analysis_records:
    print("Sample:", record["id"])
    print("Source:", record["source"])
    print("Sequence:", record["sequence"])
    print("Length:", record["length"])

    if record["valid"]:
        print("Base counts:", record["base_counts"])
        print("GC percentage:", round(record["gc_percentage"], 2))
        print("GC category:", record["gc_category"])
    else:
        print("Rejected:", record["reason"])

print("-" * 40)

```

## Inspect the resulting data structure

Use the cells below to inspect one record, retrieve one field and check how invalid and valid records differ.

## Inspection is part of programming

Printing intermediate structures is not a sign of failure. It is a basic method for checking that the program built what you intended.

Useful inspection questions include:

- How many records were created?
- Does each record have the expected ID?
- Which keys occur in a valid record?
- Which keys occur in an invalid record?
- Are derived values attached to the correct sequence?

Valid and invalid records intentionally have slightly different fields. Code that reads optional fields must therefore check `record["valid"]`, test membership, or use `.get()` appropriately.

```
In [ ]: print("Number of records:", len(analysis_records))
        print("First record:", analysis_records[0])
        print("First record ID:", analysis_records[0]["id"])
        print("Third record:", analysis_records[2])
```

## Part 14 — AI-assisted code audit

An AI assistant produced this code:

```
def base_counts(sequence):
    counts = {}
    for base in sequence:
        counts[base] += 1
    return counts
```

Do not run it immediately.

1. Predict what `+= 1` does.
2. Compare it with `+= 1`.
3. Identify what happens when a base appears more than once.
4. Identify whether lowercase input is handled.
5. Identify whether invalid DNA is detected.
6. Correct the code.
7. Write at least four assertions.

Classify each discovered problem as:

- Python syntax/operator problem;
- algorithmic problem;
- input-handling problem; or
- biological-assumption problem.

```
In [ ]: # Write your audited version and tests here.
```

## Part 15 — Independent practice

### Exercise A: Restriction-enzyme lookup

Create a dictionary mapping:

- EcoRI to GAATTC
- BamHI to GGATCC
- HindIII to AAGCTT

Then:

1. retrieve the EcoRI site;
2. ask for NotI safely;
3. loop through all enzyme–site pairs;
4. report which sites occur in AAGAATTCTTGGATCCAA .

```
In [ ]: # Write your solution here.
```

## Exercise B: Sample-status summary

Given the list below, count how many samples have each status. Your result should be a dictionary built by a loop, not typed manually.

```
In [ ]: statuses = [
        "passed", "failed", "passed", "review",
        "passed", "failed", "passed", "review"
    ]

    status_counts = {}

    # Complete the counting program.
```

## Exercise C: Records and filtering

For the samples below:

1. add sequence length to every dictionary;
2. add GC percentage;
3. print only records whose length is at least 8 and GC is at least 50%;
4. count how many records meet both conditions.

```
In [ ]: records = [
        {"id": "A", "sequence": "ATGCGTAA"},
        {"id": "B", "sequence": "AAAATTTT"},
        {"id": "C", "sequence": "GGCCGGCC"},
        {"id": "D", "sequence": "ATGC"}
    ]

    # Write your program here.
```

## Exercise D: Mini translation table

Create a dictionary containing at least:

- two codons for phenylalanine;
- two codons for alanine;
- one start codon;
- three stop codons.

Use it to translate a short DNA sequence. Clearly document:

- the starting reading position;
- treatment of incomplete codons;
- treatment of unknown codons; and
- treatment of stop codons.

## Part 16 — Formative review

Answer before running earlier examples again.

1. What does a dictionary store?
2. What is the difference between a key and a value?
3. Why must keys be unique?
4. What does `len(dictionary)` count?
5. What happens when square brackets request a missing key?
6. How does `.get()` behave for a missing key?
7. What do `.keys()`, `.values()` and `.items()` provide?
8. What does `in` check by default in a dictionary?
9. Explain `counts[base] = counts.get(base, 0) + 1`.
10. How can one dictionary represent a sequence record?
11. How do you access a dictionary stored inside another dictionary?
12. Why is a codon table suitable for a dictionary?
13. Why must translation assumptions be documented?
14. Why might `.copy()` be needed?
15. What is the difference between counting a symbol and validating it biologically?

## Mini glossary

| Term              | Beginner meaning                                                |
|-------------------|-----------------------------------------------------------------|
| dictionary        | a collection of labelled key–value pairs                        |
| key               | the label used to locate a value                                |
| value             | the information associated with a key                           |
| pair              | one key together with its value                                 |
| lookup            | retrieving a value using its key                                |
| mutable           | able to change after creation                                   |
| method            | an operation attached to an object, such as <code>.get()</code> |
| nested dictionary | a dictionary stored as a value inside another dictionary        |
| record            | related fields describing one item or sample                    |
| default value     | a fallback used when requested information is absent            |

Return to this glossary whenever the terminology makes an explanation harder to follow.

## Review answers

1. Labelled key–value pairs.
2. A key identifies an entry; its value is the associated data.
3. A key must identify one current value within that dictionary.

4. The number of key–value pairs.
5. Python raises `KeyError`.
6. It returns the value, or `None` /a supplied default when missing.
7. Views of keys, values, or key–value pairs.
8. Keys.
9. Retrieve the current count or zero, add one, and store it.
10. Use field names such as `id`, `sequence` and `organism` as keys.
11. Retrieve the outer value, then use the inner key.
12. Each codon uniquely labels an amino-acid or stop meaning in the chosen code.
13. Different assumptions can change the biological interpretation.
14. Assignment can make two names refer to the same mutable dictionary.
15. Counting records what occurred; validation decides whether it is allowed or meaningful.

## Part 17 — Take-home tracked exercise

Create:

`SBI811_Lecture6_Exercise_YourName.ipynb`

The notebook must:

1. define a list containing at least five sequence-record dictionaries;
2. include `id`, `organism` and `sequence` in every record;
3. validate each sequence;
4. add length, base counts and GC percentage to valid records;
5. add a clear rejection reason to invalid records;
6. print a formatted report;
7. use at least one function that returns a dictionary;
8. include at least five `assert` tests;
9. contain one Markdown paragraph explaining the chosen dictionary structure; and
10. contain one Markdown paragraph describing a bug and its correction.

Track the notebook in your own repository:

```
git status
git add SBI811_Lecture6_Exercise_YourName.ipynb
git commit -m "Complete SBI 811 Lecture 6 exercise"
git push origin main
```

Do not push exercise work directly to the shared class repository unless the instructor explicitly requests it.

## Lecture summary

You can now:

- create and inspect dictionaries;
- access, add and update key–value pairs;
- use safe lookup and membership testing;

- loop through dictionary keys, values and items;
- build frequency dictionaries;
- combine dictionaries with functions;
- represent biological records;
- process a list of records;
- access simple nested dictionaries; and
- use codon keys for a small translation exercise.

Lecture 7 will use these structures when reading and processing biological files such as FASTA.